Reestruturação de código com OpenMP e coordenação por evidências

Sistemas Multiagentes para Reestruturação Autônoma de Códigos

Lucas Nory Ulson · Ciência da Computação · PUCRS

Versão de trabalho · 3 de novembro de 2026

Problema: diretivas paralelas não bastam

- Uma transformação precisa produzir a saída esperada nas entradas testadas.
- Uma implementação correta pode ficar mais lenta com várias threads.
- A decisão deve apontar o código, a configuração e as medições utilizados.
- O ganho do código precisa ser separado do efeito da decisão da LLM.

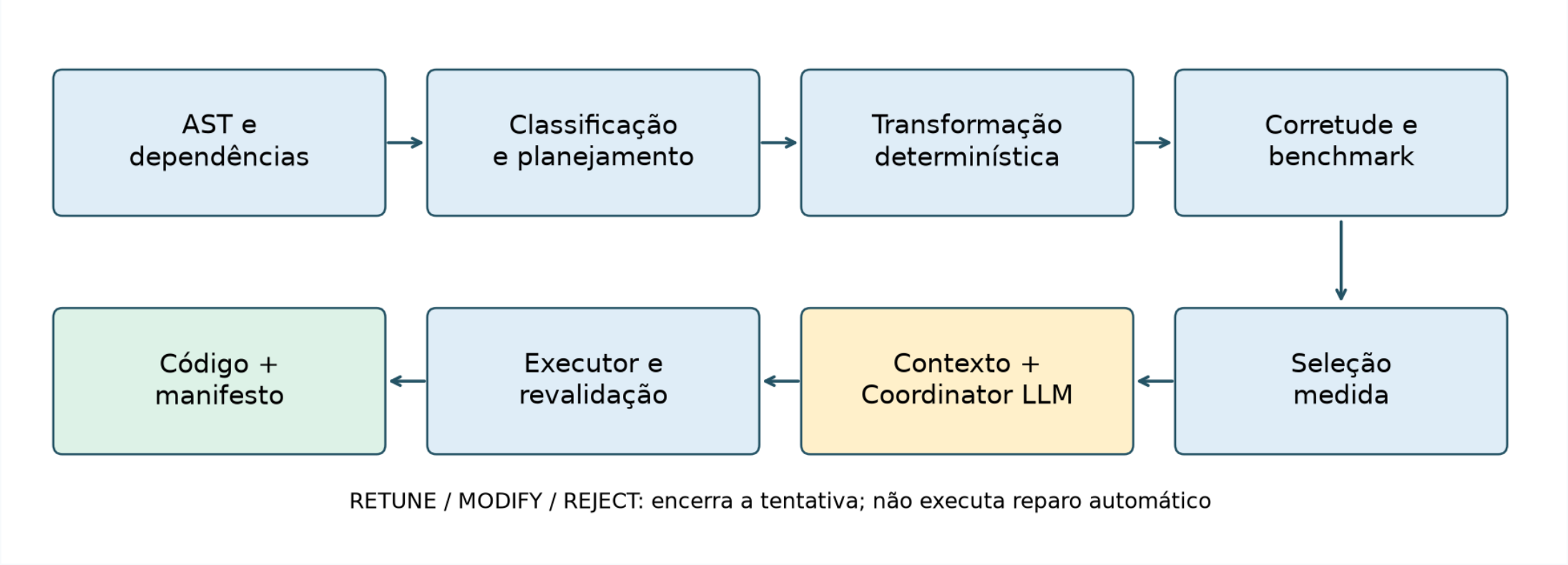
Perguntas e objetivos do recorte

- R1: como vincular artefatos e bloquear candidatos inconsistentes?
- R2: quais configurações medidas melhoram os dois baselines?
- R3: com candidato e evidências constantes, o que o Coordinator acrescenta?
- Objetivo: integrar o caminho suportado e medir decisão, resultado e custo.

Escopo: três níveis diferentes de automação

- Análises: DOALL, REDUCTION, STENCIL, WAVEFRONT e MIXED nos casos tratados.
- Caminho integrado: Dot Product C com redução escalar suportada.
- Variantes históricas: manuais ou transformações específicas sobre base existente.
- CPU/OpenMP; GPU, geração universal e reparo autônomo ficam fora da avaliação.

Arquitetura implementada



Fonte: run_pipeline.py e componentes do checkout atual.

Código medido = código autorizado = código publicado

- Cópias das entradas, SHA-256, campanha, compilador e tempos individuais.
- Seleção e contexto conferidos contra as evidências vinculadas.
- ACCEPT + configuração medida + ganho contra sequencial e OpenMP 1T.
- Recompilação e corretude fresca antes de publicar código e manifesto.

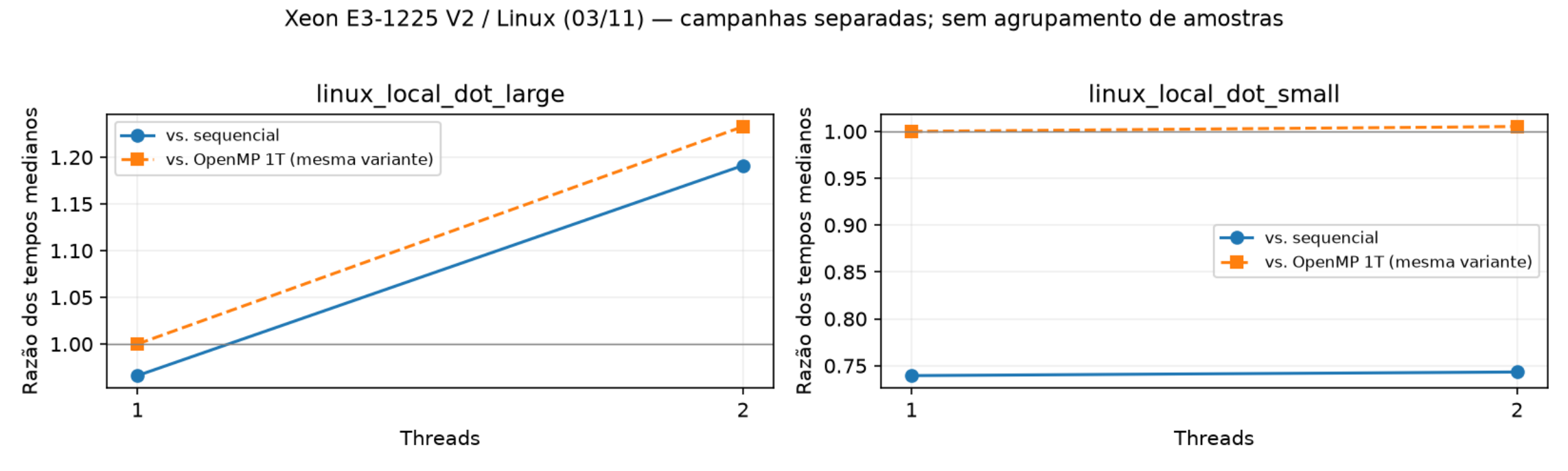
Metodologia: unidades e evidências separadas

- 19 campanhas, 161 configurações e 515 tempos individuais recuperados.
- 17 campanhas históricas sem o contrato atual completo; duas com gates locais.
- Dot Product: dois tamanhos, cinco medições externas por configuração.
- Driver de corretude separado do benchmark; sem race detector integrado.

Métricas: dois baselines, efeitos diferentes

- Ganho total: T_seq / T_p.
- Ganho de transformação: T_seq / T_1.
- Escalabilidade: T_1 / T_p; eficiência paralela: (T_1 / T_p) / p.
- Autotuning: T_1 precisa usar o mesmo bloco da variante em p threads.

Resultado local: admitir e recusar por medição



Entrada grande: ganho 1,191× em 2T; entrada pequena: paralelo não admitido.

Fonte: campanhas linux_local_dot_large e linux_local_dot_small.

Casos históricos: interpretar o ganho corretamente

- Wavefront manual: 1,412× em 1T, sem sucesso multicore.
- Wavefront tiled: bloco 64 / 4T; ganho 3,400×; escalabilidade 2,377×.
- Eficiência paralela tiled: 0,594; 0,850 usa outro denominador.
- Nussinov automático: ganho 4,106×; escalabilidade 3,005×; eficiência 0,751.

Fonte: matriz consolidada; hardware da campanha Nussinov automático não registrado.

Estudo do Coordinator: comparação pronta, resultado pendente

- Três modos: determinístico, Coordinator completo e sem análises estruturadas.
- Mesmo candidato, mesmas medições e mesmos gates nos três modos.
- Registrar concordância, aceitação, falhas, latência, tokens e chamadas.
- Rodada local: quatro inferências expiradas em 30 s; campanha INCOMPLETE.

Limitações que delimitam as conclusões

- Um kernel em dois tamanhos para o estudo pareado da LLM.
- Corretude empírica; não é prova de equivalência ou ausência geral de races.
- Histórico com lacunas de ambiente, hashes e amostras; oito CSVs inconsistentes.
- Sem busca de reparo; possível truncamento de contexto e ruído de medição.

Conclusão compatível com o estado atual

- Caminho suportado integrado, com seleção e executor de promoção auditável.
- Métricas e evidências distinguem ganho, escalabilidade e origem da variante.
- O benefício adicional do Coordinator ainda não foi demonstrado.
- Fechamento: concluir R3 e atualizar texto, resumo, gráficos e conclusão juntos.

Demonstração técnica — opcional

- Mostrar um contexto e relatórios reais vinculados ao Dot Product.
- Conferir hash, gate de corretude e configurações medidas.
- Executar um teste de recusa por alteração de arquivo, em diretório temporário.
- Separar demonstração do executor de inferência LLM e de resultado científico.

Fonte: DEMONSTRACAO.md; não depender de uma inferência terminar durante a fala.

Referências — apoio

- OpenMP ARB. OpenMP API Specification 5.2. 2021. openmp.org/spec-html/5.2/openmp.html
- Wu et al. AutoGen. 2023. arXiv:2308.08155v2.
- Nichols et al. Can Large Language Models Write Parallel Code? 2024. arXiv:2401.12554v3.
- Pouchet. PolyBench/C. Página do projeto; versão do código fixada no submódulo.

Perguntas da banca — apoio

- Por que LLM se a política já decide? É a hipótese avaliada em R3; ganho ainda aberto.
- É um sistema de vários LLMs? Não; são papéis funcionais e um Coordinator LLM.
- O código está correto para toda entrada? Os testes não estabelecem essa garantia.
- O que significam 4,106×? Ganho total; escalabilidade e eficiência usam OpenMP 1T.